Run a cloud-native workload in Docker

Session 5 – Cloud-native vs lift-and-shift

Exercise 5 · Session 5 lab · Estimated time: 75–100 min

Write your answers in the answer document `SKY2100_Exercise5_Answers.docx`. Last week you locked down a host; today you run a real application as a **container** and feel the difference between a virtual machine and a container first-hand. You will keep this Nextcloud workload: in session 6 you put it behind HTTPS, and in session 7 you configure its access control.

Caution: Run everything in your own VM. Containers here are for learning; do not expose this Nextcloud to the public internet.

1 Install Docker

▷ Install and verify Docker

(note the version in the answer document):

```
sudo apt update && sudo apt install -y docker.io
sudo docker -version
sudo docker run hello-world
```

2 Run Nextcloud as a container

▷ Pull and start the container:

```
sudo docker run -d -p 8080:80 -name nextcloud nextcloud
```

Open `http://localhost:8080` in the VM browser and complete the first-run admin setup.

▷ Inspect the running container:

`sudo docker ps`, `sudo docker logs nextcloud`.

3 Container vs VM — observe the difference

▷ Note in the answer document, and compare with your VM baseline (Exercise 2):

the approximate start-up time of the container, the ports it exposes (`docker ps`), and whether it runs its own kernel (Y/N).

▷ In one or two sentences: why did the container start so much faster than booting a VM?

Key idea: The container started in seconds because it did not boot an operating system. Your VM from Exercise 2 boots a whole kernel and its background services; the Nextcloud container reuses the host's kernel and is really just an isolated process. That is why it is small and fast, and it is also why its isolation is weaker than a VM's. A VM is separated by the hypervisor, but every container on this host shares one kernel, so a kernel-level escape reaches the host and its neighbours. The speed you liked and the security you have to think about come from the same design choice.

4 Cloud transfer: Microsoft Learn

▷ Complete "Introduction to Docker containers".

learn.microsoft.com/training/modules/intro-to-docker-containers

▷ Define the terms

you met in the lab, one line each in the answer document: image, container, registry (Docker Hub).

5 Azure reflection

▷ You ran Nextcloud with `docker run`. Azure Container Apps would run the *same* image for you. Name one thing you would no longer manage, and one new thing you would have to configure.

6 Knowledge check

1. Order these from least to most cloud change: refactor, rehost, rebuild, replatform.
   Hint: Rehost is the least change; rebuild is the most.
2. What is the key technical difference between a **VM** and a **container**?
   Hint: One runs its own kernel; one shares the host's.
3. Which kernel does a container use?
   ☐ its own ☐ a copy of the host's ☐ the shared host kernel ☐ none
   Hint: It shares the host's.
4. Name the two operating-system features that make containers possible (Comer Ch. 6).
   Hint: Namespaces give isolation, control groups (cgroups) impose limits.
5. Define **image**, **container** and **Dockerfile** in one line each.
   Hint: A read-only template, a running instance of it, and the recipe that builds the template.
6. Give *two* container-specific security risks that do not apply to a single VM.
   Hint: Shared-kernel escape, untrusted or outdated images, secrets baked into a layer.
7. Contrast a **monolith** with **microservices** in one sentence each.
   Hint: One codebase with everything, versus many small independent services.
8. State one *advantage* and one *disadvantage* of microservices (Comer Ch. 12).
   Hint: Independent scaling and fault containment — at the cost of many network calls.
9. What does a **service mesh** do, and which security control can it enforce on internal calls?
   Hint: A sidecar proxy beside each service; it can enforce mutual TLS.
10. As you move from lift-and-shift to serverless, who becomes responsible for patching the **operating system**?
    Hint: As you move toward serverless, the provider takes it over.
11. True or false: cloud-native architecture removes security risk. Justify.
    Hint: It relocates risk (to configuration, images, identity); it does not remove it.
12. (Reflection) For your Nextcloud, name one workload-security control (CSA Domain 8) you would apply first, and why.

Progress check: Discuss your answers in the group, then talk the teacher through them verbally at the next session only to track progress; nothing is handed in, signed or graded.

Solutions

Work through the lab first, then check here. Model answers are indicative — equivalent reasoning is fine.

Note: Model answers are indicative – accept equivalent reasoning. Multiple-choice answers are in **bold**.

Knowledge check

1. **Rehost → replatform → refactor → rebuild.**
2. A **VM** virtualises the *hardware* and runs its own OS kernel; a **container** virtualises the *operating system* and shares the host kernel – it is essentially an isolated process.
3. **The shared host kernel.**
4. **Namespaces** (give a process its own view of processes, network, filesystem, users) and **control groups / cgroups** (limit CPU, memory, I/O). (Comer Ch. 6, p. 73–74, verified.)
5. **Image** = a read-only template (app + dependencies, built in layers); **Container** = a running instance of an image; **Dockerfile** = a recipe that builds an image step by step.
6. Any two of: shared-kernel escape (weaker isolation than a VM); untrusted/outdated images; secrets baked into an image layer; over-privileged or root containers / host networking.
7. **Monolith** = one large program with all features in a single codebase; **microservices** = many small independent services communicating over the network. (Comer Ch. 12, p. 163–165.)
8. Advantage: independent scaling/deployment, fault containment, parallel teams. Disadvantage: many network calls (complexity/latency), harder to test and trace, more connections to secure. (Comer Ch. 12, p. 165–167, verified.)
9. A **service mesh** puts a sidecar proxy beside each service to manage service-to-service traffic; it can enforce **mutual TLS (mTLS)**, identity and policy on internal calls. (Comer Ch. 12, p. 175.)
10. **The provider** (as you move toward serverless / PaaS).
11. **False** – cloud-native *relocates* risk (from patching servers to securing configuration, images and identity); it does not remove it.
12. Reflection – e.g. harden the image, run non-root, scan for vulnerabilities, or segment the workload (CSA Domain 8).

Lab checkpoints

- The container starts in seconds because it does *not* boot an operating system – it is a process using the host kernel (contrast with the VM boot in Exercise 2).
- `docker ps` should show the `nextcloud` container mapping host `8080` to container `80`. The container does *not* run its own kernel.
- Good answers connect "faster start-up" to "no guest OS / shared kernel", not just "smaller".

References

Comer Ch. 6 – Containers (VM overhead **p. 71–72**, namespaces **p. 74**, container approach **p. 75**, Docker images/Dockerfile **p. 76–83**); Comer Ch. 12 – Microservices (monolith vs microservices **p. 163–165**, advantages/disadvantages **p. 165–167**, service mesh **p. 175**) – all verified. CSA Security Guidance v5, **Domain 8: Cloud Workload Security** (verified). Jøsang (application/workload security – syllabus, page numbers not verified). MS Learn: *Introduction to Docker containers* (verified, June 2026).